

■ EO/IR Mission Console

Project Overview & Engineering Summary

A dual-sensor operator console for real-time object detection, tracking, re-identification, cross-sensor EO/IR fusion, threat scoring, persistent mission memory, and VLM-driven mission intelligence — built to run locally on an NVIDIA RTX 5060 (8 GB, Blackwell).

1 What the System Does

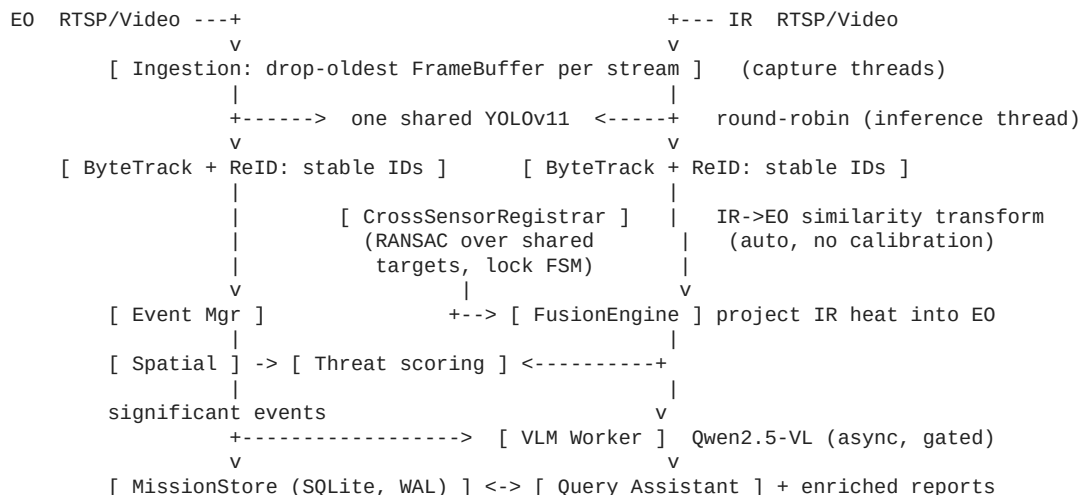
The application drives **two independent sensor streams** — a wide/zoom electro-optical (EO) channel and an infrared (IR/thermal) channel — through a shared real-time pipeline. It runs **YOLOv11** detection and **ByteTrack** tracking, gives every target a **stable identity** via an appearance re-identification layer, **automatically registers the two uncalibrated sensors** from shared targets to fuse thermal contacts into the EO view, reasons over spatial relationships and **threat level** per target, and selectively invokes a **Qwen2.5-VL** Vision-Language Model to produce natural-language mission intelligence. Everything is persisted to a **SQLite mission archive** so a mission can be queried in natural language and exported as an enriched report (Markdown, JSON, Excel, illustrated PDF).

Operator console + analysis modes

- **Live Operator Console** — two RTSP/video sources run concurrently; the EO feed is primary with an IR **picture-in-picture** (swap + fusion toggle), live boxes, stable IDs, telemetry, and right-hand operational panels (timeline, target activity, query, reporting) over the live mission.
- **Video Analysis** — a full-file pass (EO-only, IR-only, or an EO+IR pair) producing a timeline-based mission summary with an overall VLM summary.
- **Still-Image Analysis** — a single EO or IR image, or an EO+IR pair, producing EO analysis, IR analysis, and a fused assessment.

2 Architecture & Data Flow

The core design principle is to **decouple cheap, every-frame compute (detection + tracking) from the expensive VLM**, which is gated behind a rate-limited async worker. A second principle keeps the system inside the **8 GB VRAM budget**: the two sensor streams do not run two models — **one inference thread round-robins both streams through the shared YOLO** (no concurrent CUDA, no second detector). The EO stream is primary and drives memory / events / spatial / threat; IR contributes thermal and, once registered, fused contacts.



3 Component Map

~7,800 lines of Python across a modular `src/` package and a Streamlit `ui/` layer. Every stage is independently testable and swappable — the entire pipeline is unit-tested GPU-free via injectable detectors and synthetic databases.

<code>app.py / run_cli.py</code>	Operator-console UI entry point and a headless CLI runner for RTSP/video → intel.
<code>config/ + config.yaml</code>	All runtime behaviour driven from one YAML file, loaded into typed Pydantic models.
<code>ingestion/*</code>	Drop-oldest <code>FrameBuffer</code> , RTSP/webcam handler (auto-reconnect, real-FPS), video reader.
<code>detection/detector</code>	YOLOv11 (Ultralytics) wrapper with FP16 inference and warmup.
<code>tracking/tracker + reid</code>	ByteTrack association + model-free appearance ReID → stable global IDs.
<code>events/event_manager</code>	Significant-event detection (the VLM-trigger layer) + per-identity memory.
<code>reasoning/spatial</code>	Model-free geometry: proximity, following, grouping, restricted-area interactions.
<code>reasoning/registration [new]</code>	CrossSensorRegistrar — auto IR→EO similarity transform via RANSAC over shared targets.
<code>reasoning/fusion</code>	Projects IR thermal blobs into the EO frame; adaptive 8-bit hot-blob + polarity handling.
<code>reasoning/threat</code>	ThreatScorer — explainable, time-decaying per-identity threat banding (NONE...CRITICAL).
<code>memory/ [new]</code>	MissionStore — async non-blocking SQLite writer + WAL reads; full mission archive.
<code>query/ [new]</code>	QueryAssistant — NL questions → 10 whitelisted intents (no free-form SQL).
<code>vlm/qwen_vlm + vlm_worker</code>	Qwen2.5-VL 4-bit inference (EO/IR/fused prompts) behind an async, gated scheduler.
<code>intelligence/mission_intelligence</code>	Live intel feed, timeline, stats, object snapshots, report export.
<code>intelligence/mission_report [new]</code>	Enriched report (md/json/xlsx/pdf) over any archived mission, incl. grounded Q&A.
<code>intelligence/analyzers + exporters</code>	Offline Image/Video analyzers and the Excel/PDF report builders.
<code>pipeline/console [new]</code>	MissionConsole — dual-stream EO+IR backend, shared-YOLO round-robin, fusion gating.
<code>pipeline/pipeline</code>	Original single-stream real-time orchestrator (kept for single-sensor use).
<code>utils/*</code>	Logger (Loguru), shared types, image utils, drawing/HUD, NVML GPU telemetry.
<code>ui/*</code>	Console view + operational-panel modules over a cached shared model manager.

4 Significant Events (VLM Triggers)

The Event Manager maintains per-track motion histories and emits discrete, de-duplicated events. All thresholds are tunable in `config.yaml` → `events`.

<code>NEW_OBJECT</code>	a target is seen for the very first time this session

OBJECT_REIDENTIFIED [new]	a previously-seen target re-appears (recognised by ReID — no longer mis-reported as 'new')
OBJECT_LEFT	a confirmed track disappears from the scene
STOPPED	a moving target becomes stationary for N frames
DIRECTION_CHANGE	a target's heading changes beyond a threshold
PERSON_ENTER_VEHICLE	a person track vanishes adjacent to a vehicle
THERMAL_SIGNATURE	a new hot region appears (IR modality)
CROWD_FORMED	person count crosses a density threshold
THREAT_ESCALATION [new]	a target's threat band rises (highest priority — drives VLM ordering)

5 Engineering Work Delivered

Beyond the core detection / tracking / VLM pipeline, the following capabilities were designed, implemented, and smoke-tested in this codebase:

5.1 Cross-sensor EO/IR registration & fusion (calibration-free)

The operator's EO and IR are **separate, unaligned optics** (EO 66° wide + 40× zoom; IR ~12° FOV) with **no boresight, no telemetry, no calibration**, and the IR is 8-bit AGC (auto-gain, polarity can invert). A new `CrossSensorRegistrar` recovers the IR→EO **similarity transform** (scale + rotation + translation) automatically from **shared targets**: a deterministic 2-point **RANSAC over object / blob centroids** (geometry, not cross-spectral appearance — which dodges thermal intensity inversion), EMA-smoothed behind a lock state machine (UNLOCKED→ACQUIRING→LOCKED→WARM). The **recovered scale is the relative zoom**, so no zoom telemetry is needed. The reworked `FusionEngine` then projects IR thermal blobs into the EO frame (adaptive 8-bit hot-blob thresholding + white/black-hot polarity) and raises a **CONTACT_OUTSIDE_EO** slew cue when heat appears beyond the EO field of view.

5.2 Dual-stream operator console

A new `MissionConsole` backend runs **two independent sources** (EO + IR) with one inference thread **round-robinning both through the shared YOLO** — 8 GB-safe, no second model, no concurrent CUDA. EO is primary (memory / events / spatial / threat); IR contributes thermal, and fusion is gated on both streams being connected and locked. The UI was rebuilt into an **operator console**: a config sidebar, a center EO feed with IR **picture-in-picture** (swap + fusion toggles), and right-hand operational panels (Timeline / Target Activity / Query / Reporting) over the live mission.

5.3 Persistent mission memory (SQLite)

A `MissionStore` persists every mission to **SQLite** via an async non-blocking writer thread with WAL reads: missions, per-identity object lifecycle, detections, events, VLM observations, spatial interactions, and thermal/fusion events. This turns a live session into a queryable, reportable archive (SQLite chosen over Postgres deliberately — single-file, zero-ops, fits the local box).

5.4 Spatial reasoning & explainable threat scoring

A model-free `SpatialReasoner` derives **proximity, following, grouping, and restricted-area** interactions from per-track geometry (streak + cooldown noise suppression). A `ThreatScorer` synthesis layer (no new detector) combines events, interactions, fusions, and dwell into **explainable, weighted, per-second-decaying** threat factors, banding each identity NONE→LOW→MEDIUM→HIGH→CRITICAL and emitting a **THREAT_ESCALATION** event on band increase.

5.5 Natural-language Query Assistant (injection-proof)

A `QueryAssistant` answers natural-language questions over the mission archive with **no free-form SQL**: each question routes to one of **10 whitelisted intents**, each owning a single hard-coded, parameterised, read-only query — so injection is impossible by construction. The **headline answer is computed in code**; the reused Qwen2.5-VL (text-only) only reclassifies and rephrases — it *never fabricates numbers*. Works with or without the VLM (keyword fallback classifier).

5.6 Enriched reports, ReID, telemetry & robustness

- **Enriched mission reports** — `mission_report` builds a report (Markdown / JSON / Excel / illustrated PDF) over *any* archived mission: threat overview, object thumbnail gallery, merged timeline, and an optional grounded Q&A section via the Query Assistant.
- **Appearance ReID** — a model-free ReID layer (HSV histograms for EO, intensity+gradient for IR) assigns stable global IDs across ByteTrack id churn, so a returning target reads *'re-appeared'*, not *'new'* (no extra GPU model — respects the 8 GB budget).

- **Live telemetry & GPU panel** — stream resolution, true measured FPS, processing FPS, and real-time GPU/VRAM/utilisation via NVML (the in-app answer to the Task-Manager 'CUDA shows 0%' confusion).
- **RTSP FPS fix** — rejects the bogus 90000 fps H.264 RTP-clock value and measures real cadence from frame arrivals.
- **ISR prompts** — person-characteristics and suspicious-item VLM prompt templates for EO and IR.

6 Technology Stack

Detection	YOLOv11 (Ultralytics) — FP16, anchor-free, TensorRT-exportable
Tracking	ByteTrack via supervision (high+low-confidence association)
Re-ID	OpenCV histogram appearance signatures (CPU, no extra model)
EO/IR fusion	Calibration-free similarity registration (RANSAC over shared targets) + adaptive 8-bit thermal
Reasoning	Qwen2.5-VL (3B/7B) in 4-bit via transformers + bitsandbytes; in-code spatial + threat synthesis
Persistence	SQLite (WAL) async mission archive — missions, objects, events, interactions, thermal
Query	Whitelisted-intent NL assistant (no free-form SQL) over the archive
UI	Streamlit operator console — EO feed + IR PiP + operational panels
Reporting	openpyxl (Excel) · reportlab (PDF) · Markdown/JSON
Telemetry	nvidia-ml-py (NVML) GPU/VRAM, psutil
Platform	Python 3.10 · CUDA 12.8 (Blackwell) · RTX 5060 8 GB

7 Running the Application

```
# Operator console (EO + IR)
.venv\Scripts\streamlit run app.py

# Headless CLI – RTSP for 60s, export a report on exit
python run_cli.py --source rtsp://user:pass@host:554/stream --duration 60 --report

# All behaviour is tunable in config/config.yaml (detection, tracking, reid,
# vlm, events, spatial, fusion, registration, eo_sensor/ir_sensor, threat,
# memory, query, mission, ui). Edit and restart to apply.
```

8 GB VRAM note. The VLM runs in 4-bit and defaults to the 3B variant for comfortable headroom alongside YOLO; the full stack uses roughly 4.8 GB. The shared model manager keeps weights loaded once per process.